AI-Powered Customer Support Agent Copilot

With Memory, RAG, and Tool Calling

---

1. Project Title

AI-Powered Customer Support Agent Copilot with Memory, RAG, and Tool Calling

Github link -> https://github.com/ankittripathy12/cutomer_support_agent_live.git

---

2. Project Duration

Total Estimated Effort: 6-8 Hours

Phase	Time Spent
Setup and prerequisites	45–60 minutes
Memory + tool-calling implementation	90–120 minutes
Backend/API implementation (modular)	150–180 minutes
Dockerization, testing, CI/CD, EC2 deployment	90–120 minutes

---

3. Introduction

This project implements an AI-powered copilot designed to assist customer support agents in generating high-quality ticket responses efficiently.

The system integrates:

- **FastAPI** for backend APIs
- **LangChain Agent orchestration** for LLM reasoning and tool usage
- **Mem0 memory layer** for long-term contextual memory
- **ChromaDB vector database** for retrieval-augmented generation (RAG)
- **SQLite** for ticket and customer data storage
- **Streamlit** dashboard for operational usage

- **Docker + GitHub Actions + AWS EC2** for deployment

The copilot retrieves relevant policy documents, customer history, and operational data, then generates an AI draft response. The draft is editable and requires human approval before finalization. Once approved, the resolution is persisted into long-term memory to improve future responses.

This ensures accuracy, personalization, and human-in-the-loop control.

---

4. Aim

The goal of this project is to design and implement a production-style AI automation pipeline capable of:

- Ingesting and indexing support knowledge documents
- Retrieving contextual customer and policy information
- Calling structured support tools for account/ticket checks
- Generating high-quality AI draft replies
- Enabling human review and editing
- Persisting accepted resolutions into memory for future improvements

---

5. Dataset Used

This project uses a **hybrid data strategy** combining static knowledge and operational runtime data.

5.1 Knowledge Base Files

- Markdown (`.md`) and text (`.txt`) policy documents
- Stored under `knowledge_base/`
- Indexed into **ChromaDB** using chunked embeddings
- Used for Retrieval-Augmented Generation (RAG)

5.2 Operational Ticket Data

- Customer metadata
- Ticket descriptions
- Draft responses
- Stored in SQLite database: `data/support.db`
- Created dynamically via API or Streamlit dashboard

5.3 Memory Data

- Customer-level and company-level resolution memories
- Managed using **Mem0**
- Backed by Chroma vector store
- Stored in `data/chroma_mem0/`

---

6. Tools & Technologies

Core Stack

- **Programming Language:** Python 3.11
- **Backend API:** FastAPI + Uvicorn
- **Agent Framework:** LangChain (`create_agent`)
- **Checkpointer:** LangGraph

AI & Retrieval

- **LLM Provider:** Groq (`langchain-groq`)
- **Memory Layer:** Mem0
- **Vector Store (RAG):** ChromaDB

Data & Configuration

- **Database:** SQLite
- **Validation & Settings:** Pydantic + pydantic-settings

Frontend

- **Dashboard:** Streamlit

DevOps & Tooling

- **Dependency Management:** `uv`
- **Testing:** Pytest
- **Containerization:** Docker + Docker Compose
- **CI/CD:** GitHub Actions
- **Cloud Deployment:** AWS EC2 (SSH-based deployment)

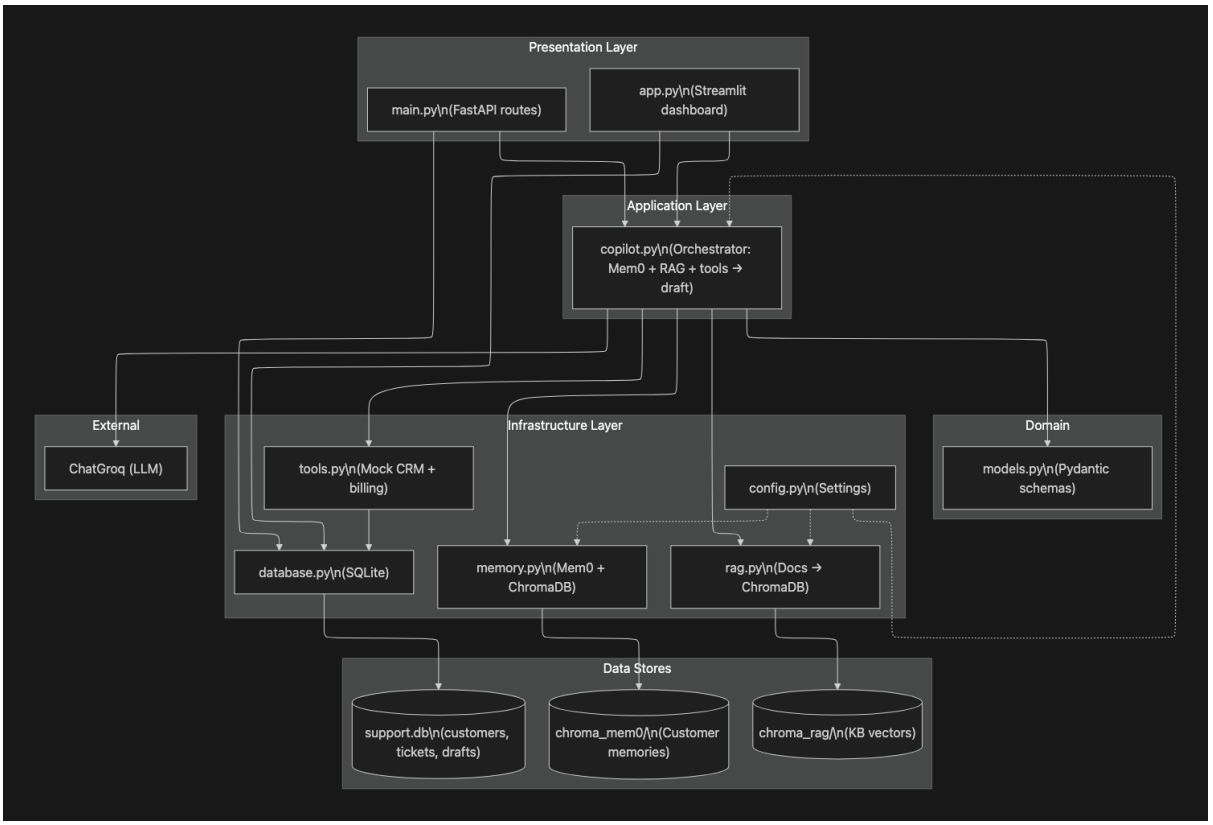
---

7. System Architecture

High-Level Flow

1. Support agent interacts with Streamlit UI
2. Request hits FastAPI backend
3. Ticket and customer data retrieved from SQLite
4. Copilot service orchestrates:
    - Memory retrieval (Mem0)
    - Knowledge base search (Chroma RAG)
    - Tool calls (plan lookup, open ticket load)
    - LLM reasoning (Groq via LangChain)
5. Draft is generated and stored
6. Agent reviews and edits draft
7. Upon acceptance:
    - Ticket marked resolved
    - Resolution persisted into memory

Architecture Overview



8. Key Modules & Implementation Steps

1. Project Setup

- `.env` configuration
- Pydantic settings management
- Directory bootstrap automation

2. Data Model Layer

- SQLite schema for:
  - Customers
  - Tickets
  - Drafts
- Repository pattern for modular access

3. Knowledge Ingestion

- File loader for `.md` and `.txt` documents
- Text chunking
- Embedding generation
- ChromaDB indexing

4. Memory Integration

- Customer scope memory
- Company scope memory
- Vector-backed semantic recall
- Resolution persistence on draft acceptance

5. Tool-Calling Layer

Custom support tools:

- Plan lookup
- Open ticket load
- Operational checks

Integrated into LangChain agent for structured function calls.

6. Copilot Orchestration

Pipeline includes:

- Memory retrieval
- RAG search
- Tool calls
- Prompt composition
- LLM reasoning
- Draft generation

7. Draft Lifecycle

- Automatic draft generation
- Manual regeneration endpoint
- Editable draft
- Acceptance workflow

8. Human-in-the-Loop

- Agent reviews draft
- Edits if necessary
- Accepts resolution
- Memory updated

9. Dashboard

- Ticket creation
- Draft review
- Memory inspection
- Regeneration controls

10. Deployment Pipeline

- Dockerized services (`api` + `dashboard`)
- Docker Compose orchestration
- GitHub Actions CI
- EC2 deployment via SSH
- Automated restart on deploy

9. Learning Outcomes

After completing this project, the learner will be able to:

- Architect modular AI backend systems
- Build REST APIs using FastAPI
- Implement RAG using ChromaDB
- Integrate long-term semantic memory using Mem0
- Implement LLM tool-calling for structured workflows
- Design human-in-the-loop AI pipelines
- Build operational dashboards with Streamlit
- Containerize AI applications using Docker
- Configure CI/CD pipelines using GitHub Actions
- Deploy AI systems to AWS EC2
- Design traceable AI reasoning flows (memory hits, tool calls, KB hits)

---

10. Optional Enhancements

Future improvements may include:

- Role-based authentication (agent/admin)
- Async job queues (Celery or RQ)
- Observability (structured logs, metrics, tracing)
- CRM/Billing real integrations
- Prompt/version evaluation tracking
- Blue-green deployment strategy
- Comprehensive test coverage (unit + integration + API contract tests)

---

12. Conclusion

This project demonstrates a real-world AI application pattern combining:

- Retrieval-Augmented Generation
- Long-term semantic memory
- Structured tool calling
- Human-in-the-loop review
- Production-ready backend architecture
- Containerized deployment

It serves as a foundation for scalable AI-powered customer support automation systems and can be extended toward enterprise-grade deployments with observability, async processing, and external integrations.